WAR-Re: Web API Recommendation with Semantic Reasoning

Anonymous submission

Abstract

With the development of cloud computing, the number of Web APIs has increased dramatically, further intensifying the demand for efficient Web API recommendation. Despite the demonstrated success of previous Web API recommendation solutions, two critical challenges persist: 1) a fixed top-N recommendation that cannot accommodate the varying API cardinality requirements of different mashups, and 2) these methods output only ranked API lists without accompanying reasons, depriving users of understanding the recommendation. To address these challenges, we propose WAR-Re, an LLM-based model for Web API recommendation with semantic reasoning for justification. WAR-Re leverages special start and stop tokens to handle the first challenge and uses two-stage training: supervised fine-tuning and reinforcement learning via Group Relative Policy Optimization (GRPO) to enhance the model's ability in both tasks. Comprehensive experimental evaluations on the ProgrammableWeb dataset demonstrate that WAR-Re achieves a gain of up to 21.59% over the state-of-the-art baseline model in recommendation accuracy, while consistently producing high-quality semantic reasons for recommendations.

1 Introduction

With the development of emerging technologies, notably cloud computing, the number of Web APIs has increased dramatically (Liu et al. 2025). For example, ProgrammableWeb reported in 2022 that its registry comprised over 24000 distinct APIs, highlighting a more than tenfold increase over the preceding decade. Web APIs enable users to leverage the functionality of other applications or services and can be easily invoked by multiple types of devices in a distributed environment. These characteristics have given rise to a new technique known as mashup, which allows developers to integrate existing Web API resources to meet complex requirements without starting from scratch (Wei et al. 2024). However, the challenge of identifying APIs that best satisfy a mashup's requirements grows with the increasing size of Web APIs. Developers particularly those without deep expertise in APIs face a combinatorial explosion when evaluating candidate APIs. Consequently, automated Web API recommendation systems have become indispensable tools, guiding developers toward high quality, compatible APIs and reducing integration effort (Wang, Xi, and Yin 2023; Wang et al. 2024).

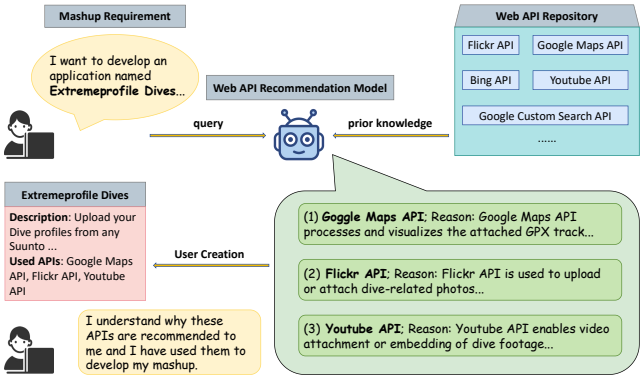

Figure 1: Example of Web API recommendation with semantic reasoning.

Numerous approaches have been proposed for Web API recommendation, ranging from content-based and collaborative filtering (CF) techniques to various hybrid strategies. Despite the existing methods' advancements, there are still two challenges: 1) The fixed top-N API recommendation fails to adapt to mashup complexity, resulting in either superfluous or insufficient recommendations. Existing approaches typically compute a matching score for each candidate API and then select the top-N APIs via simple ranking. This uniform cardinality strategy ignores that simple mashups may need only one or two APIs while complex ones require many more, thereby introducing irrelevant services in simple compositions and omitting essential ones in complex cases. 2) The lack of clear reasons for recommended APIs impairs users' understanding and trust of the recommendations. Current recommendation systems generally present users with a ranked list of APIs but offer no insight into the underlying rationale for each suggestion. Without interpretable explanations such as which functional requirements or semantic features of the mashup description triggered a particular API's inclusion, users may find the results opaque or untrustworthy which could hinder their mashup creation.

To tackle the above challenges, we propose a Web API Recommendation model with Semantic Reasoning (WAR-Re) for justification. WAR-Re utilizes TinyLlama (Zhang et al. 2024b) to understand Web APIs' descriptions and

recommend the suitable APIs for the mashup requirement. For the first challenge, WAR-Re incorporates special <API_start> and <API_stop> tokens and learns when to begin and when to end the generation of recommended APIs. For the second challenge, we propose a framework for Web API recommendation with semantic reasoning that simultaneously generates semantic reasons to explain each recommended API. Figure 1 illustrates this framework using the "Extremeprofile Dives" mashup as an example. Specifically, we begin by enriching our training corpus using DeepSeek-R1 (DeepSeek-AI 2025) which annotates each example with a natural language rationale linking the mashup description to the target API's functionality. WAR-Re is then fine-tuned in two phases: (i) supervised fine-tuning on the annotated dataset, and (ii) reinforcement learning via the Group Relative Policy Optimization (GRPO) algorithm (Shao et al. 2024) to further optimize both recommendation accuracy and reasoning quality. The main contributions of this work are summarized as follows:

- We introduce WAR-Re, a novel LLM-based model that enhances Web API recommendation with semantic reasoning and supports variable API cardinality in recommendation.
- WAR-Re is the first API recommendation model to leverage online reinforcement learning and use dedicated start and stop tokens to improve recommendation accuracy and reasoning quality.
- We demonstrate that WAR-Re outperforms previous solutions in Web API recommendation and achieves strong performance in semantic reasoning assessed by three newly proposed metrics for reasoning quality.

2 Related Work

2.1 Web API Recommendation

Web API recommendation solutions can be divided into three types: content-based, collaborative filtering (CF), and hybrid methods. Content-based Web API recommendation techniques suggest relevant APIs by leveraging salient features extracted from mashup and API descriptions (Cao et al. 2023). Shi et al. (Shi, Tang, and Liu 2019) apply an attention mechanism on LSTM to prioritize critical description segments. ServiceBERT (Wang et al. 2021b) uses BERT to encode mashup narratives. Bao et al. (Bao et al. 2017) formulate API link prediction to enhance mashup recommendations. Sang et al. (Sang, Deng, and Liao 2023) employ full-text semantic mining of developer requirements. Collaborative filtering is designed to recommend Web APIs by leveraging users' past interactions and preferences. Chen et al. (Chen, Zheng, and Lyu 2014) propose a hybrid CF approach that combines user-based and item-based strategies, incorporating QoS metrics to assess API suitability. Yao et al. (Yao et al. 2021) develop a probabilistic matrix factorization model that accounts for explicit similarity measures and latent API correlations to optimize mashup co-invocation predictions. Wang et al. (Wang et al. 2021a) integrate knowledge graph embeddings with CF, and model semantic relationships between APIs and mashups. Hybrid frameworks integrate multiple information sources to mitigate limitations of individual methods. Xia et al. (Xia et al. 2015) propose a category-aware clustering of services combined with CF to predict category-specific rankings. Qi et al. (Qi et al. 2023) construct a correlation graph to generate tailored API suggestions that meet both functional requirements and compatibility constraints. Wang et al. (Wang, Xi, and Yin 2023) develop FSFM, fusing requirement semantic vectors with representations of structural intersection.

2.2 Large Language Models with Reinforcement Learning

In recent years, reinforcement learning (RL) applied to LLMs has demonstrated remarkable success in enhancing their performance across various tasks. DeepSeek-R1 is obtained by fine-tuning DeepSeek-V3-Base on cold-start data, conducting reasoning-oriented reinforcement learning, augmenting with rejection-sampled supervised examples, and applying a final reinforcement learning pass. Qwen 2.5 (Qwen et al. 2025) implements intricate supervised fine-tuning with over 1 million samples for post-training. Its RL curriculum includes Direct Preference Optimization (DPO) and Group Relative Policy Optimization (GRPO). Kimi k1.5's (Team et al. 2025) design integrates 128k-long RL context scaling with partial rollouts for efficiency, paired with policy optimization via online mirror descent, sampling strategies, and length penalty. Seed1.5-Thinking (Seed et al. 2025) augments its RL toolkit with Value-model-based Augmented Proximal Policy Optimization (VAPO) (Yue et al. 2025) and Dynamic sAmpling Policy Optimization (DAPO) (Yu et al. 2025), leveraging these frameworks to stabilize training and enhance reasoning capabilities across complex tasks. Trained using the novel Clipped Importance Sampling Policy Optimization (CISPO) algorithm, MiniMax-M1 (MiniMax et al. 2025) demonstrates superior performance on complex software engineering tasks and long-context comprehension benchmarks. Inspired by these advances in RL for LLMs, we train WAR-Re via supervised fine-tuning followed by RL.

3 Problem Definition

3.1 Definition of Web API Recommendation with Semantic Reasoning

This study enhances Web API recommendation by integrating semantic reasoning which provides justificatory reasons for each recommended API. The task is formalized as follows: Let T^{m_i} represent the requirement description for a mashup m_i, as specified by the user. Let $A = \{a_1, \ldots, a_j, \ldots, a_S\}$ denote a repository comprising S Web APIs. Each API $a_j = \langle D^{a_j}, C^{a_j}, X^{a_j} \rangle \in A$ is defined by its functional description D^{a_j}, category C^{a_j}, and associated meta-elements X^{a_j}. Given T^{m_i} and A, our goal is twofold. First, we must identify a subset $S^{m_i} = \{a_1, \ldots, a_l, \ldots, a_s\} \subseteq A$ of size s that best satisfy the user's needs. Then, to enhance transparency and usability, each recommended API a_l is provided with an explanatory reason $r_{a_l}^{m_i}$ to justify its inclusion, yielding a set

of reasons $R^{m_i} = \{r_{a_1}^{m_i}, \ldots, r_{a_l}^{m_i}, \ldots, r_{a_s}^{m_i}\}$. Finally, using APIs from S^{m_i}, the user creates their mashup $m_i = \langle D^{m_i}, C^{m_i}, A^{m_i}, X^{m_i} \rangle$ where D^{m_i} is a textual description to introduce its functions, C^{m_i} is a set of categories to indicate its functions, A^{m_i} is a bundle of APIs to participate in the mashup, X^{m_i} is a set of meta-elements to prepare for composition. In the ideal scenario, the recommendations exactly match the chosen APIs: $A^{m_i} = S^{m_i}$.

4 Methodology

4.1 WAR-Re Architecture

Figure 2 depicts the overall architecture of WAR-Re. First, we augment the original corpus of historical mashup API calls by employing LLMs such as DeepSeek-R1 to generate explanatory reasons for why each target API is invoked, given both the mashup's requirement description and the respective API's function description. Next, WAR-Re, which is built upon TinyLlama backbone, is trained in two stages: (1) Supervised fine-tuning. We perform standard language modeling with a cross-entropy loss to train the model to predict the next token in a sequence of mashup descriptions, recommended APIs and rationale texts. After supervised fine-tuning, WAR-Re acquires foundational capabilities for API recommendation and semantic reasoning. (2) GRPO Reinforcement Learning with Low-Rank Adaptation (LoRA) (Hu et al. 2021). We then fine-tune the model via GRPO, incorporating LoRA to restrict trainable parameters to a lightweight subset, thereby reducing computational cost. After reinforcement learning, WAR-Re further enhances the performance, especially in alignment between recommendation and reasoning by ensuring more recommended APIs are accompanied by justificatory reasons.

4.2 Dataset Annotation

To train WAR-Re, we need a dataset that pairs historical mashup-API invocations with high-quality explanatory text. Historical data, however, includes only mashup descriptions and associated APIs. We address this by annotating the dataset with LLM-generated explanations. To mitigate the risk of LLM hallucinations, we provide detailed function descriptions of target APIs, ensuring reasoning is grounded in concrete information rather than speculative inferences from API names alone. Our structured prompt is given in Table 1.

Prompt	**API Recommendation Reasoning Task** Reason in English, concisely and accurately explain why the mashup uses each target API, based on the descriptions of the mashup and the target APIs. When you mention an API name, wrap it with three asterisks exactly as given in Target APIs (e.g., ***sendgrid***). **Mashup**: realtime html/css/javascript editor and advanced search engine dedicated to web development and design. **Target APIs**: blogger, google-custom-search **Target API Descriptions**: blogger: blogger data api allows client application to view update blogger content...; google-custom-search: google custom search engine api restful api allows developer get web image search result...
Answer	The mashup uses ***blogger*** to enable users to create, manage, and publish web development tutorials, code snippets, or design blogs directly within the editor, leveraging the API's ability to programmatically handle blog content. ***google-custom-search*** powers the advanced search engine, allowing users to query web development resources, documentation, or community solutions in real-time, enhancing the editor's utility.

Table 1: The prompt and example answer for dataset annotation.

4.3 First Stage Training: Supervised Fine-tuning for Foundational Capabilities

Building on the annotated dataset, we conduct supervised fine-tuning to equip WAR-Re with foundational capabilities for API recommendation and semantic reasoning. Since most API names are initially out of the tokenizer's vocabulary, we explicitly add each API as a single indivisible token (e.g., <API_sendgrid>) to prevent the tokenizer from decomposing an API name into multiple tokens and enable the model to recognize whether an API is present in or absent from the repository. Inspired by T2I-R1 (Jiang et al. 2025) which uses <img_start> for image token generation, we introduce task-specific delimiters to structure the model's output: <API_start> and <API_stop> frame the API recommendation sequence, while <REASON_start> and <REASON_stop> enclose the explanatory reasoning. This design enables the model to clearly distinguish between the two subtasks and learn precise generation boundaries. During training, the input sequence is constructed according to the template in Table 2. The template includes three key parts: the mashup's requirement description; a recommended API list enclosed by <API_start> and <API_stop>; explanatory reasons for each API, wrapped by <REASON_start> and <REASON_stop>. The mashup descriptions and recommended APIs are derived from historical records, and the explanatory reasons are annotated by LLMs. The model learns patterns in mashup descriptions to improve recommendations for similar mashups.

Let H denote the total length of the target token sequence and w_h the h-th token. The model parameterized by Φ produces a predictive distribution $P_\Phi(w_h \mid w_1, \ldots, w_{h-1})$ at each step. We adopt the standard cross-entropy loss over the entire sequence:

$$\mathcal{L}_\Phi = -\frac{1}{H} \sum_{h=1}^{H} \log P_\Phi(w_h \mid w_1, \ldots, w_{h-1}) \qquad (1)$$

Here, the term $\log P_\Phi(w_h \mid w_1, \ldots, w_{h-1})$ measures the log-likelihood of the true token w_h under the model's

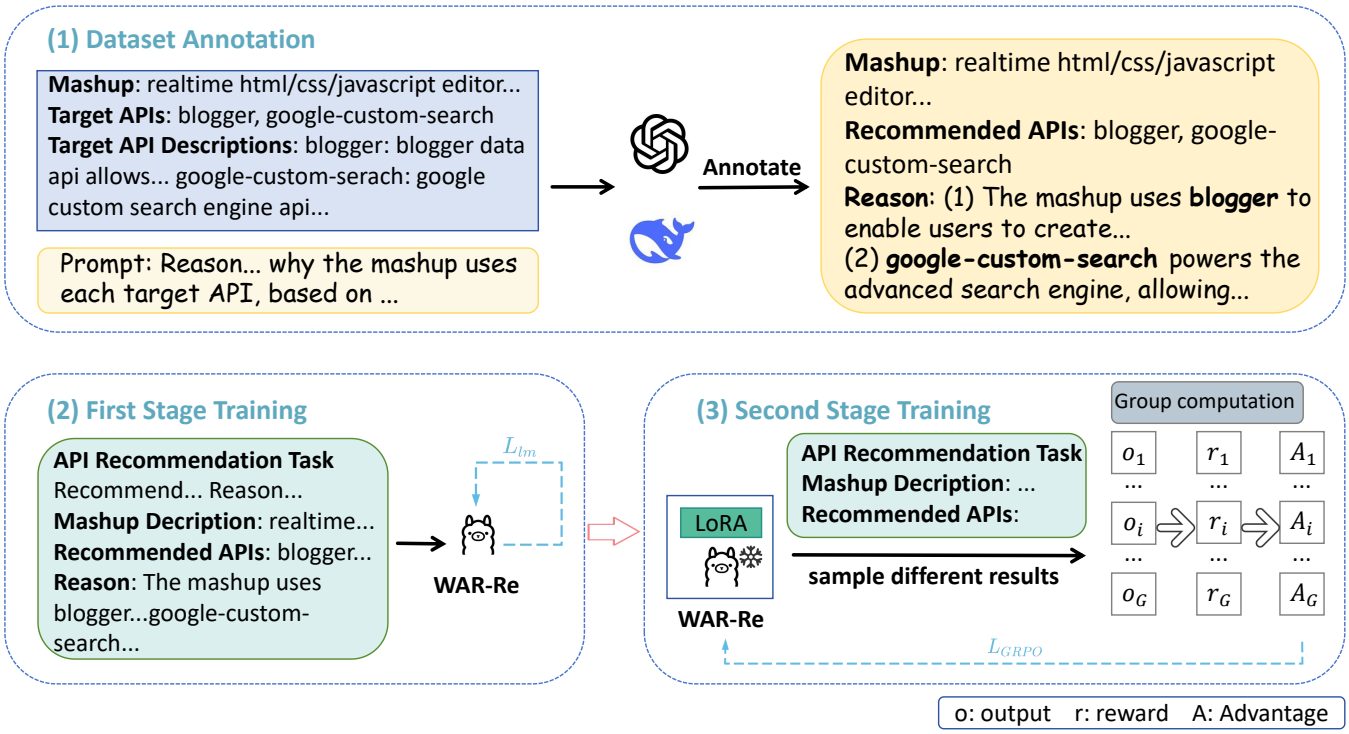

Figure 2: The architecture of WAR-Re.

First Stage Prompt	**API Recommendation Task** Recommend APIs for the mashup according to its description and give the reason for each recommendation. **Mashup Description**: realtime html/css/-javascript editor and advanced search engine dedicated to web development and design. **Recommended APIs**: <API_start> <API_blogger> <API_google-custom-search> <API_stop> **Reason**: <REASON_start> The mashup uses <API_blogger> to enable users to create, manage, and publish web ... <API_google-custom-search> powers the advanced search engine, allowing users to query web development ...<REASON_stop>
Second Stage Prompt	**API Recommendation Task** Recommend APIs for the mashup... **Mashup Description**: realtime html... **Recommended APIs**:

Table 2: The prompt for the two-stage training of WAR-Re.

predictive distribution. Negating and averaging these log-likelihoods across all positions h in the sequence encourages the model to prioritize correct tokens at each step. This formulation ensures the model learns foundational capabilities of generating syntactically valid sequences that respect the structure imposed by special tokens (e.g., transitioning from <API_start> to API tokens to <API_stop>), selecting APIs relevant to the mashup description, and producing coherent reasons that align with the recommended APIs.

4.4 Second Stage Training: GRPO with LoRA for Performance Enhancement

Overall Introduction After the first stage training, the model gains foundational recommendation and reasoning capabilities. However, supervised fine-tuning is limited by reliance on static training data and lacks mechanisms to explore new API combinations beyond historical patterns. To further enhance performance, we adopt GRPO as its exploration mechanisms and group-based comparative feedback encourages the model to explicitly learn which outputs better satisfy evaluation metrics. For each input question q omitting recommended APIs and reasons as given in Table 2, GRPO samples a group of outputs $\{o_1, o_2, \ldots, o_G\}$ from the old policy $\pi_{\theta_{\text{old}}}$ and then optimizes the policy model by maximizing the following objective:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{\substack{q\sim P(Q),\\ \{o_i\}\sim\pi_{\theta_{\text{old}}}}} \Bigg[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|} \min\Big\{\frac{\pi_\theta(o_{i,t}\mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}\mid q, o_{i,<t})}\hat{A}_{i,t},\ \text{clip}\Big(\frac{\pi_\theta}{\pi_{\theta_{\text{old}}}}, 1-\varepsilon, 1+\varepsilon\Big)\hat{A}_{i,t}\Big\} - \beta\, D_{\text{KL}}\big[\pi_\theta \| \pi_{\text{ref}}\big]\Bigg]. \quad (2)$$

Here ε and β are hyper-parameters, and $\hat{A}_{i,t}$ is the advantage computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}\big(\{r_1, r_2, \cdots, r_G\}\big)}{\text{std}\big(\{r_1, r_2, \cdots, r_G\}\big)}. \tag{3}$$

GRPO incorporates regularization by adding the KL divergence between the learned policy and the reference policy directly into the loss, thereby sidestepping any added complexity in computing $\hat{A}_{i,t}$. In our implementation, the reference policy π_{ref} is set to the old policy $\pi_{\theta_{\text{old}}}$, ensuring alignment with the policy update framework.

To reduce computational cost, we adopt LoRA in our GRPO training. For a given weight matrix $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$, LoRA employs a low-rank decomposition to modify the weights, expressed as $\mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{BA}$, where $\mathbf{B} \in \mathbb{R}^{d\times r}$ and $\mathbf{A} \in \mathbb{R}^{r\times k}$, with the rank $r \ll \min(d, k)$. During the training process, $\mathbf{W}_0$ remains fixed and does not receive gradient updates, while the matrices $\mathbf{A}$ and $\mathbf{B}$ are trainable. Thus, in our GRPO stage we only optimize the LoRA parameters $\{\mathbf{A}, \mathbf{B}\}$.

Design of the Reward Function Our overall reward consists of two components: (i) Recommendation quality and (ii) Reasoning quality.

For the recommendation reward, we employ standard information retrieval metrics to assess API recommendation performance. As s denotes the number of recommended APIs output by the model, we use "@s" to indicate that these metrics are computed over all APIs recommended by the model. Precision@s refers to the ratio of the number of real hits in the recommended APIs to the number of recommended APIs. Recall@s is defined as the ratio of the number of real hits in the recommended APIs to the number of all real APIs. Furthermore, the F1 score as the harmonic mean of Precision and Recall, balances these two metrics. Since Precision@s and Recall@s do not take into account the order in the results, we use additional evaluation metrics: Mean Average Precision (MAP) and Normalized Discounted Cumulative Gain (NDCG). NDCG is calculated from DCG by normalizing IDCG which is the ideal DCG. Average Precision (AP) calculates Precision at each position, and averages these values over all matching positions. MAP is obtained by averaging AP across all mashups. In practice, the batch size is set to 1 during the second-stage training. As a result, the reward function at this stage directly uses AP@s computed for each individual mashup. The recommendation reward is given as:

$$\begin{aligned}\text{recommendation reward} = 0.4 * \text{F1} + 0.1 * \text{Precision} \\ +0.1 * \text{Recall} + 0.2 * \text{NDCG} + 0.2 * \text{AP}.\end{aligned} \tag{4}$$

Since F1 encapsulates both Precision and Recall, we assign it a greater weight; however, we retain small weights for individual metrics to reward high performance in specific aspects.

For the reasoning reward, we introduce Reasoning Precision (RP) and Reasoning Recall (RR) to evaluate the quality of reasoning. RP quantifies the proportion of all provided reasons that correspond to APIs actually recommended by the model.

$$\text{RP} = \frac{|\text{APIs}_{\text{recommend}} \cap \text{APIs}_{\text{reason}}|}{|\text{APIs}_{\text{reason}}|}. \tag{5}$$

RR quantifies the proportion of all APIs recommended by the model that are accompanied by a valid reason.

$$\text{RR} = \frac{|\text{APIs}_{\text{recommend}} \cap \text{APIs}_{\text{reason}}|}{|\text{APIs}_{\text{recommend}}|}. \tag{6}$$

RP and RR are equally weighted, so the reasoning reward is formulated as:

$$\text{reasoning reward} = 0.5 * \text{RP} + 0.5 * \text{RR}. \tag{7}$$

The overall reward is the sum of the recommendation reward and reasoning reward, as both components carry equal significance in our model design.

Statistics	**Value**
Number of APIs	1647
Number of Mashups	8217
Number of Categories	499
APIs per Mashup	2.091
Words in description per Mashup	17.019
Words in description per API	44.296

Table 3: Statistics of the dataset.

5 Experiments And Analysis

To evaluate the effectiveness of WAR-Re, we need to answer the following questions:

RQ1: How does WAR-Re's accuracy in Web API recommendation compare to that of baseline models?

RQ2: How effective is WAR-Re in semantic reasoning?

RQ3: What is the impact of the second stage training strategy on WAR-Re's overall performance?

RQ4: What is the influence of the special start and stop tokens on WAR-Re's overall performance?

RQ5: Does incorporating the semantic reasoning task influence WAR-Re's ability to recommend Web APIs?

5.1 Dataset and Setting

We use the Mashup and API data from the public dataset (Wu et al. 2021) crawled from ProgrammableWeb. The dataset contains 8217 mashups and 1647 APIs. Detailed statistics of the dataset are presented in Table 3. To evaluate model performance, we randomly partition the mashup dataset into training, validation, and test subsets in a 3:1:1 ratio, while retaining the full API repository. For the first-stage fine-tuning, the learning rate is set to 1e-5, for the second-stage training, the learning rate is set to 5e-6. Each stage is trained for 20 epochs. The experimental setup utilizes 4 Nvidia V100 GPUs (32GB).

5.2 Baseline Models

We compare WAR-Re against four representative baselines:

- **SPR**: The Service Profile Reconstruction model uses topic modeling on mashup descriptions and APIs to derive topic structures and recommend APIs accordingly (Zhong et al. 2018).
- **RWR**: The Random Walk with Restart model encodes mashup contexts as category entities in a compact knowledge-graph schema, then applies RWR to rank APIs by relevance (Wang, Wu, and Hsu 2019).
- **MTFM**: MTFM uses a multi-layer CNN to embed mashups and APIs, concatenates their embeddings for collaborative filtering, and employs multi-task learning with mashup category prediction (Wu et al. 2021).
- **BERT-CM**: BERT-CM uses BERT to encode mashup descriptions and is trained through a linear classification layer (Zhang et al. 2024a).

5.3 Evaluation Metrics

As mentioned before, to assess our Web API recommendation component, we employ standard information retrieval metrics: Precision@s (P@s), Recall@s (R@s), NDCG@s (N@s), and MAP@s (M@s). Unlike many prior works that fix N to evaluate Top-N recommendations, our model outputs on average between 1.7 and 2.6 APIs per query depending on the temperature. Therefore, for baseline methods we report metrics at $N = 1$ and $N = 3$, while for our approach we evaluate at $s = 1$ and at a dynamically determined cutoff s. Higher values of these metrics correspond to better recommendation accuracy and ranking quality.

For the reasoning component, in addition to Reasoning Precision (RP) and Reasoning Recall (RR) defined previously, we introduce Reasoning Score (RS) generated from LLMs. Specifically, we provide the LLM with the mashup description, the set of recommended APIs and their function descriptions, and the generated reasons, then ask it to assign a score in $[0, 1]$ based on: (1) The degree to which the reason addresses the mashup requirements; (2) The consistency of the reason with the official API descriptions; (3) The overall clarity and correctness of the explanation. The exact prompt template and an example LLM response are shown in Table 4.

5.4 Main Results

RQ1: Performance of WAR-Re on Web API recommendation and comparison with baseline models WAR-Re outperforms all other models across all metrics. Under greedy decoding, WAR-Re generates an average of 1.79 APIs per output, with 52.22% of these outputs fully matching the ground truth and a maximum case where 31 recommended APIs are entirely correct. As shown in Table 5, which compares WAR-Re with four baselines, WAR-Re achieves 19.56% and 21.59% improvements over BERT-CM in Precision@1 and Recall@1 respectively. Even with its compact average output size, WAR-Re still exceeds BERT-CM's Recall@3 by 15.85%. Additionally, it demonstrates substantially better ranking quality as measured by NDCG@s and MAP@s. Built on pretrained TinyLlama, WAR-Re effectively captures long-range dependencies and subtle semantic connections between mashup requirements and API descriptions. Its variable cardinality recommendation directly overcomes fixed top-N limitations by adapting to mashup complexity, avoiding excess APIs for simple cases and ensuring adequacy for complex ones.

Prompt	**API Reason Evaluation Task** You are given a mashup description, recommended APIs, their corresponding descriptions, and the reason for the recommendation. You are asked to evaluate the reason for the recommendation and give a score ranging from 0 to 1 by analyzing: (1) How well the reason aligns with the mashup requirements (2) How well the reason aligns with the API descriptions (3) The overall quality and accuracy of the reason The final score should be the average assessment of all the recommended APIs. If any API doesn't have a reason, it should be given 0 definitely. The API names in the reason are already enclosed in angle brackets as given in Recommended APIs, for example: `<API_sendgrid>`. Please provide only the numerical score (0 to 1) as your response. **Mashup**: Inflo collects day-to-day information stream like email appointment instant... **Recommended APIs**: flickr, blogger, youtube, last.fm, facebook, twitter, yahoo-mail, bing **Recommended APIs Description**: flickr: flickr api used retrieve photo...; blogger: blogger data api allows client application view... **Reason**: `<API_flickr>` : To retrieve... `<API_blogger>` : To fetch blog posts...
Answer	0.69

Table 4: The prompt and example answer for Reasoning Score.

RQ2: Performance of WAR-Re in semantic reasoning WAR-Re consistently generates high-quality semantic reasons for its API recommendations. We evaluate the quality of the generated rationales using three LLMs, DeepSeek-R1 (DS), Llama 3.3, and GPT-4.1 Mini, so as to produce a more robust RS. Across all recommended APIs, 87.88% are accompanied by a corresponding rationale. Table 6 summarizes the detailed metrics of WAR-Re computed over each mashup instance. For RP, 84.75% of instances achieve the maximum score 1, yielding an average RP of 0.908. For RR, 85.48% of instances score 1, with an average RR of 0.912. With respect to RS, the proportion of instances scoring between 0.9 and 1.0 stands at 61.39% under DeepSeek-R1, 65.77% under Llama 3.3, and 66.14% under GPT-4.1 Mini, with an overall average RS of 0.790. Our findings indicate that WAR-Re produces high-quality, semantically justified

Methods	P@1	P@3/s	R@1	R@3/s	N@3/s	M@3/s
SPR	0.419	0.247	0.287	0.437	0.544	0.514
RWR	0.429	0.263	0.310	0.482	0.577	0.543
MTFM	0.604	0.294	0.411	0.526	0.661	0.646
BERT-CM	0.680	0.334	0.477	0.612	0.730	0.715
WAR-Re	0.813	0.767	0.580	0.709	0.834	0.827

Table 5: Performance comparison of baseline models on the task of Web API recommendation.

Models	Recommendation					
	P@1	P@s	R@1	R@s	N@s	M@s
WAR-Re(nG)	0.812	0.748	0.579	0.712	0.832	0.825
WAR-Re(nS)	0.811	0.661	0.578	0.726	0.836	0.827
WAR-Re(Rec)	0.773	0.727	0.550	0.676	0.798	0.790
WAR-Re(re)	—	0.796	—	0.706	0.820	0.842
WAR-Re(nre)	—	0.432	—	0.240	0.439	0.448
WAR-Re	0.813	0.767	0.580	0.709	0.834	0.827

Models	Reasoning					
	RP	RR	RS			
	—	—	DS	Llama	GPT	AVG
WAR-Re(nG)	0.885	0.891	0.760	0.779	0.765	0.768
WAR-Re(nS)	0.869	0.873	0.740	0.769	0.745	0.751
WAR-Re	0.908	0.912	0.785	0.798	0.787	0.790

Table 6: Performance comparison of ablation models on dual tasks.

reasons for the majority of its API recommendations.

5.5 Ablation Study

To answer RQ3-RQ5, we design the ablation models and conduct the ablation study.

RQ3: Effectiveness of the second stage training The second-stage GRPO training improves both recommendation accuracy and reasoning quality. We compare the full WAR-Re model against an ablated variant, WAR-Re(nG), which omits the second stage GRPO training. WAR-Re(nG) recommends an average of 1.95 APIs per mashup with 83.32% of these accompanied by rationales, whereas the full WAR-Re model issues 1.79 recommendations per mashup, 87.88% of which include rationales. Table 6 reports the core performance metrics on the Web API recommendation task. By filtering out recommendations without rationales, Precision@s improves by 2.54%. Likewise, RP and RR increase by 2.60% and 2.36% respectively, and the proportion of instances achieving both RP=1 and RR=1 rises from 80.07% to 83.77%. Finally, the average Reasoning Score increases by 2.86%. These results confirm that the second-stage GRPO training substantially enhances both the accuracy of Web API recommendation and the quality of semantic reasoning.

RQ4: Effectiveness of the special tokens The inclusion of special tokens markedly enhances control over API and rationale generation, boosting overall performance. By removing the special tokens `<API_start>`, `<API_stop>`, `<REASON_start>` and `<REASON_stop>`, we derive WAR-Re(nS). Without the constraint of these special tokens, WAR-Re(nS) recommends an average of 2.57 APIs per mashup, but only 77.95% of them include an accompanying rationale. Consequently, although WAR-Re(nS) achieves a slightly higher R@s than WAR-Re, its Precision, RP, RR and RS all decline. Specifically, as shown in Table 6, P@s drops 13.82%, RP drops 4.30%, RR drops 4.28% , average RS drops 4.94%. These special tokens enable WAR-Re to better segment and control the initiation and termination of both API recommendation and semantic reasoning.

RQ5: Impact of Semantic Reasoning on API Recommendation Effectiveness Integrating reasoning substantially elevates API recommendation effectiveness across all metrics. To assess how the inclusion of reasoning affects recommendation performance, we introduce an ablation variant, WAR-Re(Rec), which omits the reasoning component.WAR-Re(Rec) is trained in two stages to ensure a fair comparison. In the first stage, the model is presented with the same API function descriptions utilized by the reasoning component of WAR-Re. In the second stage, we fine-tune WAR-Re(Rec) solely on the recommendation task. The prompts are exemplified in the appendix. To quantify the impact of reasoning on API recommendation quality, we separately evaluate metrics for recommendations accompanied by reasons (WAR-Re(re)) versus those without (WAR-Re(nre)). As reported in Table 6, integrating reasoning into WAR-Re yields notable improvements over WAR-Re(Rec) with 5.50% higher P@s, 4.88% higher R@s, 4.51% higher N@s, and 4.68% higher M@s. Recommendations furnished with explanations outperform unaccompanied suggestions by over 84.26% across all metrics. These findings demonstrate that requiring the model to justify each suggestion adds accountability: semantically defensible API choices are favored, reducing spurious outputs and improving reliability.

6 Conclusion

In this study, we propose WAR-Re, a novel LLM-based model that enhances Web API recommendation with semantic reasoning and supports variable API cardinality for diverse mashup complexities. We first use DeepSeek-R1 to generate explanations for historical mashup–API calls in the dataset and attach them as annotations. Then WAR-Re is trained via two stages: supervised fine-tuning followed by GRPO reinforcement learning with LoRA. Experimental results demonstrate WAR-Re's superior performance in recommendation accuracy over state-of-the-art models and high quality in semantic reasoning as validated by quantitative metrics and LLM evaluations. Ablation studies confirm the effectiveness of GRPO training, the special start and stop tokens and the mutual benefits of integrating reasoning with recommendation tasks. Future work will focus on increasing the scale of the underlying language model to further broaden WAR-Re's applicability in real world scenarios.

References

Bao, Q.; Zhang, J.; Duan, X.; Ramachandran, R.; Lee, T. J.; Zhang, Y.; Xu, Y.; Lee, S.; Pan, L.; Gatlin, P.; and Maskey, M. 2017. A Fine-Grained API Link Prediction Approach Supporting Mashup Recommendation. In *Proc. of the IEEE International Conference on Web Services (ICWS)*, 220–228. Honolulu, HI, USA.

Cao, B.; Peng, M.; Zhang, L.; Qing, Y.; Tang, B.; Kang, G.; and Liu, J. 2023. Web Service Recommendation via Integrating Heterogeneous Graph Attention Network Representation and FiBiNET Score Prediction. *IEEE Transactions on Services Computing*, 16(5): 3837–3850.

Chen, X.; Zheng, Z.; and Lyu, M. R. 2014. QoS-Aware Web Service Recommendation via Collaborative Filtering. *Web Services Foundations*, 563–588.

DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2021. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

Jiang, D.; Guo, Z.; Zhang, R.; Zong, Z.; Li, H.; Zhuo, L.; Yan, S.; Heng, P.-A.; and Li, H. 2025. T2I-R1: Reinforcing Image Generation with Collaborative Semantic-level and Token-level CoT. arXiv:2505.00703.

Liu, M.; Wu, G.; Xu, H.; Wang, J.; Xu, X.; and Wang, Z. 2025. AdaFlow: Learning and Utilizing Workflows for Enhanced Service Recommendation in Dynamic Environments. *IEEE Transactions on Services Computing*, 18(2): 572–585.

MiniMax; :; Chen, A.; Li, A.; Gong, B.; Jiang, B.; Fei, B.; Yang, B.; Shan, B.; Yu, C.; et al. 2025. MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention. arXiv:2506.13585.

Qi, L.; Lin, W.; Zhang, X.; Dou, W.; Xu, X.; and Chen, J. 2023. A Correlation Graph Based Approach for Personalized and Compatible Web APIs Recommendation in Mobile APP Development. *IEEE Transactions on Knowledge and Data Engineering*, 35(6): 5444–5457.

Qwen; :; Yang, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; et al. 2025. Qwen2.5 Technical Report. arXiv:2412.15115.

Sang, C.; Deng, X.; and Liao, S. 2023. Mashup-Oriented Web API Recommendation Via Full-Text Semantic Mining of Developer Requirements. *IEEE Transactions on Services Computing*, 16(4): 2755–2768.

Seed, B.; :; Chen, J.; Fan, T.; Liu, X.; Liu, L.; Lin, Z.; Wang, M.; Wang, C.; Wei, X.; et al. 2025. Seed1.5-Thinking: Advancing Superb Reasoning Models with Reinforcement Learning. arXiv:2504.13914.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.

Shi, M.; Tang, Y.; and Liu, J. 2019. Functional and Contextual Attention-Based LSTM for Service Recommendation in Mashup Creation. *IEEE Transactions on Parallel and Distributed Systems*, 30(5): 1077–1090.

Team, K.; Du, A.; Gao, B.; Xing, B.; Jiang, C.; Chen, C.; Li, C.; Xiao, C.; Du, C.; et al. 2025. Kimi k1.5: Scaling Reinforcement Learning with LLMs. arXiv:2501.12599.

Wang, X.; Liu, X.; Liu, J.; Chen, X.; and Wu, H. 2021a. A novel knowledge graph embedding based API recommendation method for Mashup development. *World Wide Web*, 24(3): 869–894.

Wang, X.; Wu, H.; and Hsu, C.-H. 2019. Mashup-Oriented API Recommendation via Random Walk on Knowledge Graph. *IEEE Access*, 7: 7651–7662.

Wang, X.; Xi, M.; Li, Y.; Pan, X.; Wu, Y.; Deng, S.; and Yin, J. 2024. SEHGN: Semantic-Enhanced Heterogeneous Graph Network for Web API Recommendation. *IEEE Transactions on Services Computing*, 17(5): 2836–2849.

Wang, X.; Xi, M.; and Yin, J. 2023. Functional and structural fusion based web api recommendations in heterogeneous networks. In *Proc. of the IEEE International Conference on Web Services (ICWS)*, 91–96. Chicago, IL, USA.

Wang, X.; Zhou, P.; Wang, Y.; Liu, X.; Liu, J.; and Wu, H. 2021b. ServiceBERT: A Pre-trained Model for Web Service Tagging and Recommendation. In *Service-Oriented Computing: 19th International Conference, ICSOC 2021, Virtual Event, November 22–25, 2021, Proceedings*, 464–478. Berlin, Heidelberg: Springer-Verlag. ISBN 978-3-030-91430-1.

Wei, C.; Fan, Y.; Jia, Z.; and Zhang, J. 2024. Cross-View Graph Alignment for Mashup Recommendation. *IEEE Transactions on Services Computing*, 17(5): 2151–2164.

Wu, H.; Duan, Y.; Yue, K.; and Zhang, L. 2021. Mashup-Oriented Web API Recommendation via Multi-Model Fusion and Multi-Task Learning. *IEEE Transactions on Services Computing*, 15(6): 3330–3343.

Xia, B.; Fan, Y.; Tan, W.; Huang, K.; Zhang, J.; and Wu, C. 2015. Category-Aware API Clustering and Distributed Recommendation for Automatic Mashup Creation. *IEEE Transactions on Services Computing*, 8(5): 674–687.

Yao, L.; Wang, X.; Sheng, Q. Z.; Benatallah, B.; and Huang, C. 2021. Mashup Recommendation by Regularizing Matrix Factorization with API Co-Invocations. *IEEE Transactions on Services Computing*, 14(2): 502–515.

Yu, Q.; Zhang, Z.; Zhu, R.; Yuan, Y.; Zuo, X.; Yue, Y.; Dai, W.; Fan, T.; et al. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476.

Yue, Y.; Yuan, Y.; Yu, Q.; Zuo, X.; Zhu, R.; Xu, W.; Chen, J.; Wang, C.; et al. 2025. VAPO: Efficient and Reliable Reinforcement Learning for Advanced Reasoning Tasks. arXiv:2504.05118.

Zhang, C.; Qin, S.; Wu, H.; and Zhang, L. 2024a. Cooperative Mashup Embedding Leveraging Knowledge Graph for Web API Recommendation. *IEEE Access*, 12: 49708–49719.

Zhang, P.; Zeng, G.; Wang, T.; and Lu, W. 2024b. TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385.

Zhong, Y.; Fan, Y.; Tan, W.; and Zhang, J. 2018. Web Service Recommendation With Reconstructed Profile From Mashup Descriptions. *IEEE Transactions on Automation Science and Engineering*, 15(2): 468–478.